TRƯỜNG ĐẠI HỌC FPT

****

FPT UNIVERSITY

LAB211 – OOP WITH JAVA LAB

MOUNTAIN HIKING CHALLENGE REGISTRATION REPORT

INSTRUCTOR'S NAME: LÊ VÕ MINH THƯ

Submission Day: 9 June 2026

SE203056	**Nguyễn Phú Khương**

Contents

1. Project Introduction

1.1. Problem Description

The project "J1.L.P0027 - Mountain Hiking Challenge Registration" is a console-based application built in Java, designed to manage student registrations for the Mountain Hiking Challenge. The program allows an Operator to perform CRUD (Create, Read, Update, Delete) operations on the registration list, as well as search, filter by campus, generate statistics by mountain peak, and save/load data to/from files.

This application was developed as part of the LAB211 - Basic Java course at FPT University, aimed at practicing Object-Oriented Programming (OOP), file I/O handling, input data validation, and organizing source code with a clear class structure.

1.2. Lab Requirements

According to the LAB211 assignment, students must fulfill the following requirements:

- Build a Java console application to manage mountain hiking registrations for FPT students.
- The menu consists of 9 features: New Registration, Update, Display, Delete, Search, Filter, Statistics, Save, Exit.
- Apply OOP: use classes, interfaces, extend ArrayList, implement Serializable and Comparable.
- Validate input data using Regex: Student ID, Phone, Email, Name, Campus Code.
- Calculate registration fee: default 6,000,000 VND, 35% discount for Viettel or VNPT carriers.
- Read the mountain list from the MountainList.csv file.
- Save registration data to registrations.dat (binary) and export to registrations.csv.
- Prompt for confirmation when exiting if there is unsaved data.
- Code must be at least 200 LOC (Lines of Code).

1.3. Technologies Used

No.	Technology	Description
1	Java SE	Primary programming language (Java 8)
2	Java I/O	Read/write CSV files, serialization (.dat)
3	Java Collections	ArrayList, HashMap, Collections.sort()
4	Regex (Pattern)	Input data format validation
5	Serialization	Store Student objects to binary file
6	IntelliJ IDEA	Development IDE

2. System Design

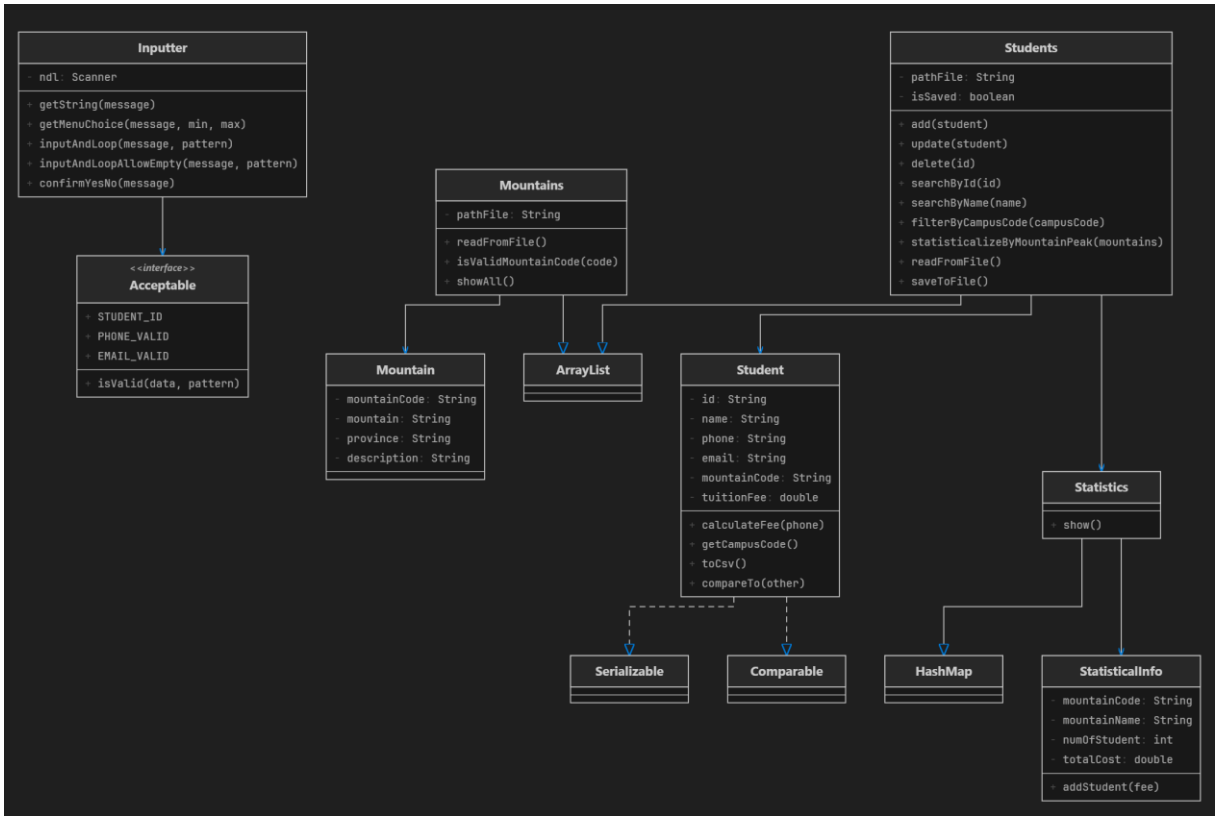
2.1. Project Structure

The project is organized with the following directory structure:

```
Project1_MountainHiking/
|-- src/
|   |-- Main.java              (Main class, menu and orchestration)
|   |-- Acceptable.java         (Interface with Regex constants)
|   |-- Inputter.java           (Console input handling class)
|   |-- Mountain.java           (Mountain entity class)
|   |-- Mountains.java          (Mountain list, reads from CSV)
|   |-- Student.java            (Student entity class)
|   |-- Students.java           (Student list, CRUD, file I/O)
|   |-- StatisticalInfo.java     (Statistics for one mountain peak)
|   |-- Statistics.java          (Aggregated statistics table)
|-- MountainList.csv            (Data: 13 mountain peaks)
|-- registrations.dat           (Binary file for registrations)
|-- registrations.csv           (CSV export of registrations)
|-- Class Diagram.drawio        (Class diagram)
|-- FlowChart.drawio            (Overall flowchart)
|-- FlowChart_PerFunc.drawio    (Flowchart per feature)
```

2.2. Class Diagram

Below is the Class Diagram illustrating the relationships between classes in the system. The classes are designed following OOP principles with inheritance, interface implementation, and references between objects.



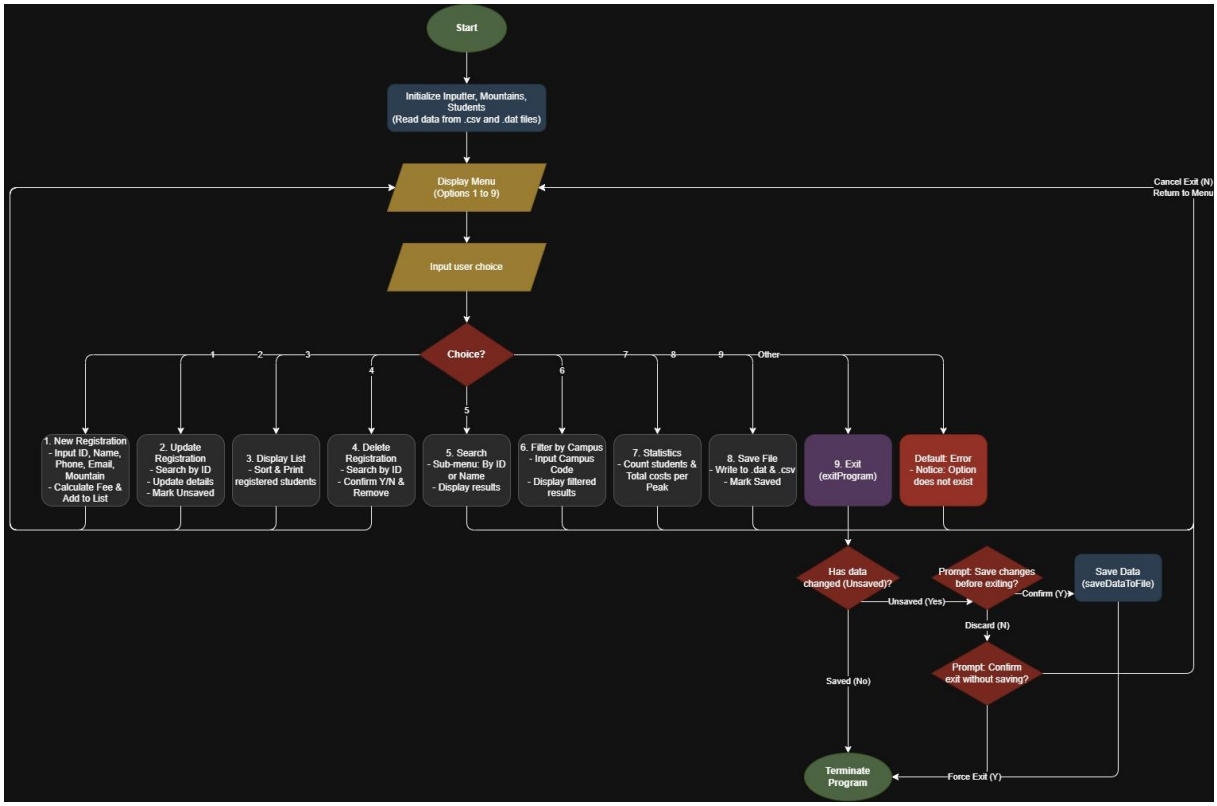
(Insert image: Class Diagram - System class relationships)

Key relationships in the Class Diagram:

- Student implements Serializable: enables serializing Student objects for .dat file storage.
- Student implements Comparable<Student>: enables comparing and sorting students by ID.
- Students extends ArrayList<Student>: the student list inherits from ArrayList.
- Mountains extends ArrayList<Mountain>: the mountain list inherits from ArrayList.
- Statistics extends HashMap<String, StatisticalInfo>: the statistics table uses HashMap.
- Inputter uses Acceptable: calls isValid() method for data validation.
- Students references Student; Statistics references StatisticalInfo.

2.3. Overall Flowchart

The overall flowchart describes the main workflow of the program from startup to exit. The program operates in a menu-driven loop.



(Insert image: Overall Flowchart - Main program workflow)

Workflow description:

1. Step 1: Startup - Load the mountain list from MountainList.csv.
2. Step 2: Load registration data from registrations.dat (if it exists).
3. Step 3: Display the main menu with 9 options.
4. Step 4: The user enters a choice (validated 1-9).
5. Step 5: Execute the corresponding feature.
6. Step 6: Mark data as unsaved if any changes were made (Add/Update/Delete).
7. Step 7: Return to Step 3 until the user chooses Exit (9).
8. Step 8: If there is unsaved data, prompt to save -> End.

3. Detailed Class Descriptions

3.1. Interface Acceptable

Acceptable is an interface that contains Regex (Regular Expression) constants used for input data validation, and a static method isValid() to check whether a string matches a given regex pattern.

Constant	Regex Pattern	Meaning
STUDENT_ID	^(?i)(SE|HE|DE|QE|CE)\d{6}$	Student ID: 2-char campus code + 6 digits
CAMPUS_CODE	^(?i)(SE|HE|DE|QE|CE)$	Campus code: SE/HE/DE/QE/CE
NAME_VALID	^[A-Za-zAa-yy\s]{2,20}$	Name: 2-20 letters and spaces only
PHONE_VALID	^0\d{9}$	Phone: starts with 0, exactly 10 digits
VIETTEL_VALID	^(032|033|...|086)\d{7}$	Viettel carrier phone (discounted fee)
VNPT_VALID	^(081|082|...|094)\d{7}$	VNPT carrier phone (discounted fee)
EMAIL_VALID	^[A-Za-z0-9+_.-]+@...+\.[A-Za-z]{2,}$	Valid email format
YES_NO_VALID	^[YyNn]$	Confirmation: Y or N

Method isValid(String data, String pattern): Uses Pattern.matches() to check whether the data string matches the pattern. Returns true if it matches, false otherwise.

3.2. Class Inputter

Inputter is a utility class for handling keyboard (console) input. This class ensures all input data is validated before being returned.

Method	Description	Notes
getString(message)	Displays a prompt and reads a string	Returns trimmed string
getInt(message)	Reads an integer, loops until valid	Uses INTEGER_VALID regex
getDouble(message)	Reads a double, loops until valid	Uses DOUBLE_VALID regex
getMenuChoice(msg, min, max)	Reads a menu choice within	Loops if out of range

	[min, max]	
inputAndLoop(msg, pattern)	Reads input and validates with regex	Does not allow empty input
inputAndLoopAllowEmpty(msg, pattern)	Same as above but allows empty (Enter)	Used during Update
confirmYesNo(message)	Asks for Y/N confirmation	Returns true if Y

3.3. Class Mountain

Mountain represents a mountain peak in the program. Each Mountain object stores the mountain code, name, province, and description.

Attribute	Data Type	Description
mountainCode	String	Mountain code (e.g., 1, 2, 3...)
mountain	String	Mountain name (e.g., Ham Rong Mountain)
province	String	Province/city where the mountain is located
description	String	Detailed description of the mountain

Special methods: toString() displays information in columnar format, equals() compares by mountainCode (case-insensitive), hashCode() is based on the uppercase mountainCode.

3.4. Class Mountains

Mountains extends ArrayList<Mountain> and manages the entire list of mountain peaks. It automatically reads data from MountainList.csv upon initialization.

Method	Description
Mountains()	Default constructor, reads MountainList.csv
Mountains(pathFile)	Constructor with custom file path
get(mountainCode)	Finds a Mountain by code (case-insensitive)
isValidMountainCode(code)	Checks whether a mountain code exists
dataToObject(text)	Converts a CSV line into a Mountain object
readFromFile()	Reads the entire CSV file, skips header and blank lines
showAll()	Displays the mountain list in a table with header

3.5. Class Student

Student represents a student who registered for the hiking challenge. This class implements Serializable (for binary file storage) and Comparable<Student> (for sorting by ID).

Attribute	Data Type	Description
id	String	Student ID (e.g., SE203056)
name	String	Student full name
phone	String	Phone number (10 digits, starts with 0)
email	String	Email address
mountainCode	String	Registered mountain peak code
tuitionFee	double	Registration fee (VND)

Important constants:

- DEFAULT_FEE = 6,000,000 VND - Default registration fee.
- DISCOUNT_RATE = 0.35 (35%) - Discount rate for Viettel/VNPT phone numbers.

Fee calculation logic (calculateFee):

- If the phone number belongs to Viettel or VNPT -> Fee = 6,000,000 x (1 - 0.35) = 3,900,000 VND.
- If the phone number belongs to another carrier -> Fee = 6,000,000 VND (no discount).

Note: When calling setPhone(), the registration fee is automatically recalculated. This ensures the fee is always accurate when the phone number is updated.

3.6. Class Students

Students extends ArrayList<Student> and manages the entire registration list. It supports CRUD operations, search, filter, statistics, and file I/O.

Method	Description	Notes
add(student)	Adds a student, checks for duplicate ID	Calls markUnsaved() on success
update(student)	Updates a student by ID	Calls markUnsaved() on success
delete(id)	Deletes a student by ID	Calls markUnsaved() on success
searchById(id)	Exact search by ID (case-insensitive)	Returns Student or null
searchByName(name)	Search by name (partial, case-insensitive)	Returns List<Student>
filterByCampusCode(campus)	Filters by first 2 characters of ID	e.g., SE, HE, DE...
showAll()	Displays the entire list	Sorted by ID
showAll(list)	Displays a sub-list	Used for search/filter results
readFromFile()	Reads .dat file (ObjectInputStream)	Called in constructor
saveToFile()	Saves .dat file + exports CSV	Marks as saved afterward
statisticalizeByMountainPeak()	Generates statistics by peak	Creates Statistics object

3.7. Class StatisticalInfo

StatisticalInfo stores statistics for a specific mountain peak, including the mountain code, mountain name, number of registered students, and total fee.

Attribute	Data Type	Description
mountainCode	String	Mountain peak code
mountainName	String	Mountain peak name
numOfStudent	int	Number of registered students
totalCost	double	Total registration fee (VND)

Method addStudent(fee): Increments numOfStudent by 1 and adds fee to totalCost.

3.8. Class Statistics

Statistics extends HashMap<String, StatisticalInfo> and performs statistical analysis of registration count and total fees grouped by mountain peak.

Statistics logic (statisticalize):

9. Iterates through the entire student list.
10. For each student, retrieves the mountainCode.
11. If no StatisticalInfo exists for that mountainCode, creates a new one and looks up the mountain name from Mountains.
12. Calls addStudent(fee) to accumulate the count and total.
13. Displays a statistics table with columns: Code, Peak Name, Number of Participants, Total Cost.

3.9. Class Main

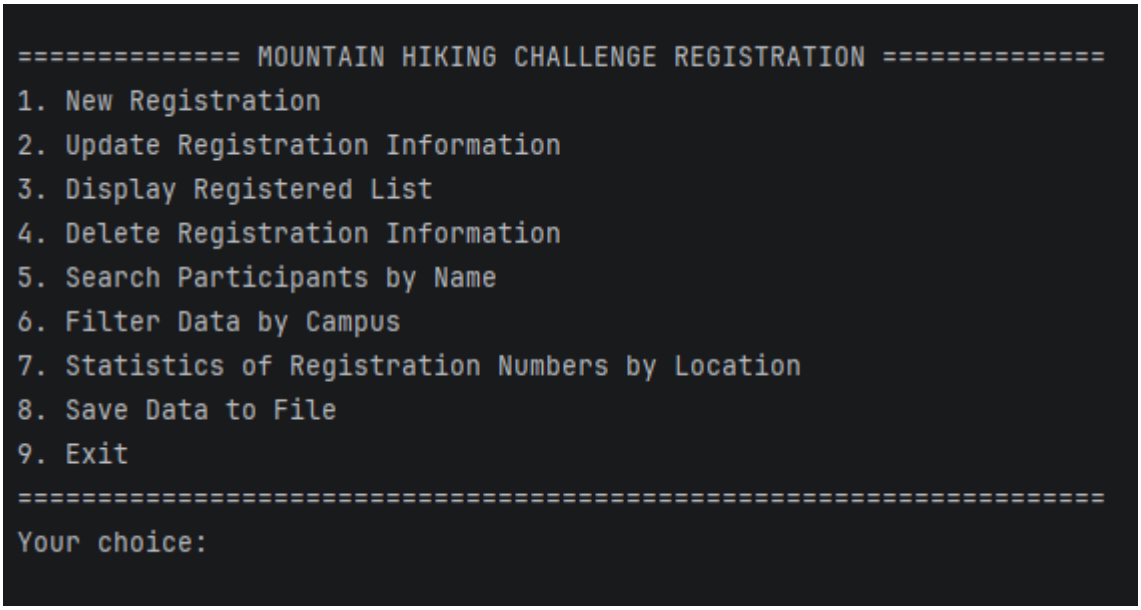
Main is the primary orchestration class containing the main() method and static methods corresponding to each menu feature. Main initializes 3 core objects: Inputter (input handling), Mountains (mountain list), and Students (student list).

Method	Menu	Feature
addNewRegistration()	1	Register a new student
updateRegistration()	2	Update registration information
displayRegisteredList()	3	Display the registered list
deleteRegistration()	4	Delete a registration
searchByName()	5	Search by ID or name
filterByCampus()	6	Filter by campus
showStatistics()	7	Statistics by mountain peak
saveDataToFile()	8	Save data to file
exitProgram()	9	Exit the program

4. User Guide for Each Feature

4.1. Main Menu

When the program starts, the main menu is displayed. The user enters a number from 1 to 9 to select the corresponding feature. If an invalid input is entered (letters, out-of-range numbers, special characters), the program prompts for re-entry without crashing.



(Insert image: Main Menu interface upon program startup)

Menu features:

1. New Registration: Register a new student for the hiking challenge.

2. Update Registration Information: Update information of an already registered student.

3. Display Registered List: Display the entire registration list in table format.

4. Delete Registration Information: Delete a student's registration.

5. Search Participants by Name: Search for students by ID or name.

6. Filter Data by Campus: Filter the list by campus (SE/HE/DE/QE/CE).

7. Statistics of Registration Numbers by Location: Statistics of count and total fee by mountain peak.

8. Save Data to File: Save data to .dat file and export CSV.

9. Exit: Exit the program (prompts to save if there is unsaved data).

4.2. Feature 1: New Registration

Description: Allows entering new student information to register for the hiking challenge.

Steps:

14. Step 1: Select option 1 from the main menu.
15. Step 2: Enter Student ID in the format: SE/HE/DE/QE/CE + 6 digits (e.g., SE203056). If the ID already exists, an error is displayed and re-entry is required.
16. Step 3: Enter Name: 2-20 characters, letters and spaces only.
17. Step 4: Enter Phone: 10 digits, starting with 0.
18. Step 5: Enter Email: valid email format (e.g., example@gmail.com).
19. Step 6: The list of 13 mountain peaks is displayed. Enter the desired mountain code (1-13).
20. Step 7: The system automatically calculates the fee: 3,900,000 VND (Viettel/VNPT) or 6,000,000 VND (other).
21. Step 8: A success message with the registration fee is displayed.

```
======================================================================
Your choice: 1
---- New Registration ----
Student ID [SE/HE/DE/QE/CE + 6 digits]: SE200000
Name (2-20 chars): Nguyen Phu Khuong
Phone (10 digits, starting with 0): 0363561629
Email: khuongsosad@gmail.com
----------------------------------------------------------------------
Code  | Mountain                | Province        | Description
----------------------------------------------------------------------
1     | Ham Rong Mountain       | Lao Cai         | Located near the
2     | Doi Bo Mountain         | Lao Cai         |
3     | Pha Luong Mountain      | Son La          | Nearly 2000m abov
4     | Hon Vuon Mountain       | Hue             |
5     | Da Do Mountain          | Ninh Thuan      |
6     | Da Bia Mountain         | Phu Yen         |
7     | Chu Hreng Mountain      | Kon Tum         |
8     | Lang Biang Mountain     | Lam Dong        |
9     | Ta Nang Mountain        | Lam Dong        |
10    | Cam Mountain            | An Giang        |
11    | Thi Vai Mountain        | Vung Tau        |
12    | Dinh Mountain           | Vung Tau        |
13    | Co Tien Mountain        | Khanh Hoa       | The mountain has
----------------------------------------------------------------------
Mountain code: 1
Registration added successfully. Tuition fee: 3,900,000 VND
```

(Insert image: New Registration screen - entering complete student information)

Error handling during registration:

- Invalid ID format -> "Invalid format, please re-enter."
- Duplicate ID -> "Student ID already exists. Please try again."
- Invalid name, phone, or email -> "Invalid format, please re-enter."
- Invalid mountain code -> "Invalid mountain code. Please choose a code from the list."

```
==================================================================
Your choice: 1
---- New Registration ----
Student ID [SE/HE/DE/QE/CE + 6 digits]: JJ938
Invalid format, please re-enter.
Student ID [SE/HE/DE/QE/CE + 6 digits]: SE203056
Student ID already exists. Please try again.
Student ID [SE/HE/DE/QE/CE + 6 digits]: SE203333
Name (2-20 chars): Khuong
Phone (10 digits, starting with 0): 0382729
Invalid format, please re-enter.
Phone (10 digits, starting with 0): 0363561629
Email: kkkk@kkkkkk
Invalid format, please re-enter.
Email: khuong@gmai.com
------------------------------------------------------------------
Code  | Mountain                | Province        | Descripti
------------------------------------------------------------------
1     | Ham Rong Mountain       | Lao Cai         | Located n
2     | Doi Bo Mountain         | Lao Cai         |
3     | Pha Luong Mountain      | Son La          | Nearly 20
4     | Hon Vuon Mountain       | Hue             |
5     | Da Do Mountain          | Ninh Thuan      |
6     | Da Bia Mountain         | Phu Yen         |
7     | Chu Hreng Mountain      | Kon Tum         |
8     | Lang Biang Mountain     | Lam Dong        |
9     | Ta Nang Mountain        | Lam Dong        |
10    | Cam Mountain            | An Giang        |
11    | Thi Vai Mountain        | Vung Tau        |
12    | Dinh Mountain           | Vung Tau        |
13    | Co Tien Mountain        | Khanh Hoa       | The mount
------------------------------------------------------------------
Mountain code: 111
Invalid mountain code. Please choose a code from the list.
Mountain code: 421321
Invalid mountain code. Please choose a code from the list.
Mountain code: sdsad
Invalid mountain code. Please choose a code from the list.
Mountain code: 4
Registration added successfully. Tuition fee: 3,900,000 VND
```

(Insert image: Error message screen when entering invalid data)

4.3. Feature 2: Update Registration Information

Description: Updates the registration information of an existing student.

Steps:

22. Step 1: Select option 2 from the main menu.
23. Step 2: Enter the Student ID to update.
24. Step 3: If the ID does not exist -> "This student has not registered yet." -> Return to menu.
25. Step 4: Display the student's current information.
26. Step 5: For each field (Name, Phone, Email, Mountain Code): enter a new value or press Enter to keep the current value.
27. Step 6: If the Phone is changed, the registration fee is automatically recalculated.
28. Step 7: Display the updated information.

```
================================================================
Your choice: 2
---- Update Registration ----
Student ID to update: SE203056
Current information:
-------------------------------------------------------------------------------------------------
StudentID  | Name               | Phone       | Email                  | PeakCode  |          Fee
-------------------------------------------------------------------------------------------------
SE203056   | Nguyen Phu Khuong  | 0363561629  | khuong@gmail.com       | 1         |    3,900,000
-------------------------------------------------------------------------------------------------
Press Enter to keep the old value.
```

(Insert image: Update screen - displaying old information and entering new values)

```
Your choice: 2
---- Update Registration ----
Student ID to update: SE203056
Current information:
-------------------------------------------------------------------------------------------------
StudentID  | Name               | Phone       | Email                  | PeakCode  |         Fee
-------------------------------------------------------------------------------------------------
SE203056   | Nguyen Phu Khuong  | 0363561629  | khuong@gmail.com       | 1         |   3,900,000
-------------------------------------------------------------------------------------------------
Press Enter to keep the old value.
New name (current: Nguyen Phu Khuong): Nguyễn Phú Khương
New phone (current: 0363561629): 0363561622
New email (current: khuong@gmail.com): phukhuong@gmail.com
New mountain code (current: 1, Enter to keep): 3
Updated successfully.
-------------------------------------------------------------------------------------------------
StudentID  | Name               | Phone       | Email                  | PeakCode  |         Fee
-------------------------------------------------------------------------------------------------
SE203056   | Nguyễn Phú Khương  | 0363561622  | phukhuong@gmail.com    | 3         |   3,900,000
-------------------------------------------------------------------------------------------------
```

(Insert image: Successful update result)

Note: The "press Enter to keep current value" feature is very convenient when only 1-2 fields need to be modified without re-entering everything.

4.4. Feature 3: Display Registered List

Description: Displays the entire list of registered students in table format, sorted by Student ID in ascending order.

Steps:

29. Step 1: Select option 3 from the main menu.
30. Step 2: The table is displayed with columns: StudentID, Name, Phone, Email, PeakCode, Fee.

- If no one has registered: "No students have registered yet."

```
9. Exit
================================================================
Your choice: 3
---- Registered Students ----
-------------------------------------------------------------------------------------------------------
StudentID  | Name               | Phone       | Email                   | PeakCode  |          Fee
-------------------------------------------------------------------------------------------------------
SE200000   | Nguyen Phu Khuong  | 0363561629  | khuongsosad@gmail.com   | 1         |    3,900,000
SE203056   | Nguyễn Phú Khương  | 0363561622  | phukhuong@gmail.com     | 3         |    3,900,000
SE203333   | Khuong             | 0363561629  | khuong@gmai.com         | 4         |    3,900,000
-------------------------------------------------------------------------------------------------------
```

(Insert image: Display screen showing the full registration list in table format)

4.5. Feature 4: Delete Registration

Description: Removes a student's registration from the list.

Steps:

31. Step 1: Select option 4 from the main menu.
32. Step 2: Enter the Student ID to delete.
33. Step 3: If the ID does not exist -> error message -> return to menu.
34. Step 4: Display the student's detailed information for confirmation.
35. Step 5: Prompt for confirmation: "Are you sure you want to delete this registration? (Y/N)".
36. Step 6: If Y -> delete and display "The registration has been successfully deleted."
37. Step 7: If N -> "Deletion cancelled." -> return to menu.

```
================================================================
Your choice: 4
---- Delete Registration ----
Student ID to delete: SE203333
Student details:
-------------------------------------------------┼-----------------------------------------------
StudentID  | Name               | Phone       | Email                   | PeakCode  |          Fee
-------------------------------------------------------------------------------------------------------
SE203333   | Khuong             | 0363561629  | khuong@gmai.com         | 4         |    3,900,000
-------------------------------------------------------------------------------------------------------
```

(Insert image: Delete screen - confirmation before deletion)

```
Your choice: 4
---- Delete Registration ----
Student ID to delete: SE203333
Student details:
-------------------------------------------------------------------------------------------------
StudentID  | Name                | Phone        | Email                    | PeakCode  |          Fee
-------------------------------------------------------------------------------------------------
SE203333   | Khuong              | 0363561629   | khuong@gmai.com          | 4         |    3,900,000
-------------------------------------------------------------------------------------------------
Are you sure you want to delete this registration? (Y/N): Y
The registration has been successfully deleted.
```

(Insert image: Result after successful deletion)

4.6. Feature 5: Search Participants

Description: Search for students by exact Student ID or by partial name (case-insensitive).

Steps:

38. Step 1: Select option 5 from the main menu.
39. Step 2: A sub-menu appears: 1. Search by Student ID / 2. Search by Name.
40. Step 3: If option 1: Enter Student ID -> exact search -> display result or "No one matches the search criteria!"
41. Step 4: If option 2: Enter (partial) name -> find all students whose name contains the input (case-insensitive) -> display result table.

```
==================================================================
Your choice: 5
---- Search Registration ----
1. Search by Student ID
2. Search by Name
Your choice: 1
Student ID: SE203056
-------------------------------------------------------------------------------------------------
StudentID  | Name                | Phone        | Email                    | PeakCode  |          Fee
-------------------------------------------------------------------------------------------------
SE203056   | Nguyễn Phú Khương   | 0363561622   | phukhuong@gmail.com      | 3         |    3,900,000
-------------------------------------------------------------------------------------------------
```

(Insert image: Search by ID screen - result found)

```
Your choice: 5
---- Search Registration ----
1. Search by Student ID
2. Search by Name
Your choice: ng
Please choose a number from 1 to 2.
Your choice: 2
Enter (partial) name to search: kh
Matching Students:
-------------------------------------------------------------------------------------------------
StudentID  | Name               | Phone       | Email                   | PeakCode  |         Fee
-------------------------------------------------------------------------------------------------
SE200000   | Nguyen Phu Khuong  | 0363561629  | khuongsosad@gmail.com   | 1         |   3,900,000
SE203056   | Nguyễn Phú Khương  | 0363561622  | phukhuong@gmail.com     | 3         |   3,900,000
-------------------------------------------------------------------------------------------------
```

(Insert image: Search by Name screen - multiple matching results)

```
========================================================================
Your choice: 5
---- Search Registration ----
1. Search by Student ID
2. Search by Name
Your choice: 1
Student ID: SE200300
No one matches the search criteria!
```

```
========================================================================
Your choice: 5
---- Search Registration ----
1. Search by Student ID
2. Search by Name
Your choice: 2
Enter (partial) name to search: nam
No one matches the search criteria!
```

(Insert image: Search screen - no results found)

4.7. Feature 6: Filter by Campus

Description: Filters the student list by campus code (first 2 characters of the Student ID).

Campus Code	Campus Name
CE	FPT Can Tho
DE	FPT Da Nang
HE	FPT Ha Noi
SE	FPT Ho Chi Minh
QE	FPT Quy Nhon

Steps:

42. Step 1: Select option 6 from the main menu.
43. Step 2: Enter the campus code (e.g., SE).
44. Step 3: Display the list of students under that campus, or "No students have registered under this campus." if none found.

```
Your choice: 6
---- Filter by Campus ----
Campus codes: CE - Can Tho | DE - Da Nang | HE - Ha Noi | SE - Ho Chi Minh | QE - Quy Nhon
Enter campus code: SE
Registered Students Under Campus (SE):
-------------------------------------------------------------------------------------------------
StudentID  | Name               | Phone       | Email                   | PeakCode  |         Fee
-------------------------------------------------------------------------------------------------
SE200000   | Nguyen Phu Khuong  | 0363561629  | khuongsosad@gmail.com   | 1         |   3,900,000
SE203056   | Nguyễn Phú Khương  | 0363561622  | phukhuong@gmail.com     | 3         |   3,900,000
-------------------------------------------------------------------------------------------------
```

(Insert image: Filter by Campus screen - displaying results)

4.8. Feature 7: Statistics by Mountain

Description: Generates statistics of the number of registered students and total fee per mountain peak.

Steps:

45. Step 1: Select option 7 from the main menu.
46. Step 2: The statistics table is displayed with columns: Code, Peak Name, Number of Participants, Total Cost.

- If there is no data: "No registration data available for statistics."

```
Your choice: 7
---- Statistics by Mountain Peak ----
Statistics of Registration by Mountain Peak:
-------------------------------------------------------------------------
Code  | Peak Name               | Number of Participants |      Total Cost
-------------------------------------------------------------------------
1     | Ham Rong Mountain       | 1                      |       3,900,000
3     | Pha Luong Mountain      | 1                      |       3,900,000
-------------------------------------------------------------------------
```

(Insert image: Statistics by Mountain screen - results table)

4.9. Feature 8: Save Data to File

Description: Saves all registration data to 2 files:

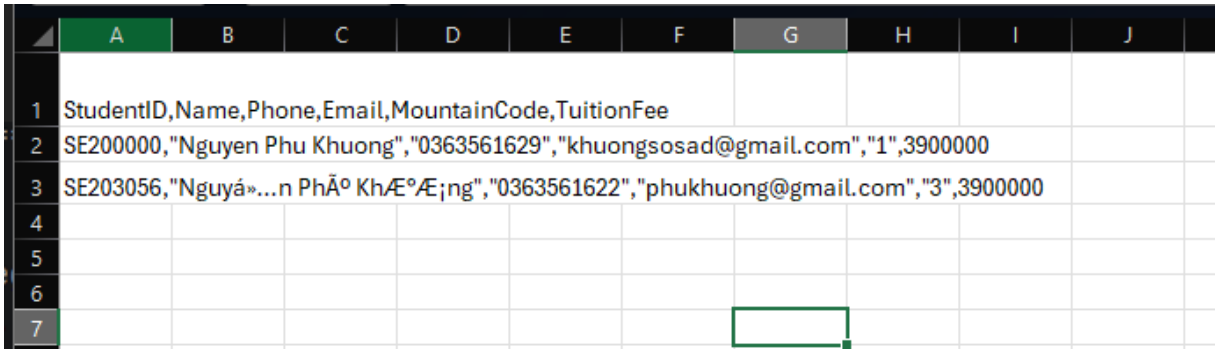
- registrations.dat - Binary file, uses ObjectOutputStream to serialize ArrayList<Student>.
- registrations.csv - Text CSV file, easily opened with Excel. Columns: StudentID, Name, Phone, Email, MountainCode, TuitionFee.

Steps:

47. Step 1: Select option 8 from the main menu.
48. Step 2: The system automatically saves and displays "Registration data has been successfully saved."

```
================================================================
Your choice: 8
---- Save Data to File ----
Registration data has been successfully saved to `registrations.dat`.
CSV report has been exported to `registrations.csv`.

============== MOUNTAIN HIKING CHALLENGE REGISTRATION ==============
```

(Insert image: Successful data save screen)



	A	B	C	D	E	F	G	H	I	J
1	StudentID,Name,Phone,Email,MountainCode,TuitionFee									
2	SE200000,"Nguyen Phu Khuong","0363561629","khuongsosad@gmail.com","1",3900000									
3	SE203056,"Nguyá»...n PhÃº KhÆ°Æ¡ng","0363561622","phukhuong@gmail.com","3",3900000									
4										
5										
6										
7										

(Insert image: Contents of registrations.csv opened in Excel or Notepad)

4.10. Feature 9: Exit Program

Description: Exits the program with unsaved data checking.

Exit workflow:

49. Select option 9 from the main menu.
50. If there is NO unsaved data -> Exit immediately, display "Goodbye!".
51. If there IS unsaved data:
52. a. Prompt: "You have unsaved changes. Do you want to save before exiting? (Y/N)"
53. b. If Y -> Save to file -> Exit.
54. c. If N -> Prompt again: "Are you sure you want to exit without saving? (Y/N)"
55. d. If Y -> Exit without saving.
56. e. If N -> Return to menu (do not exit).

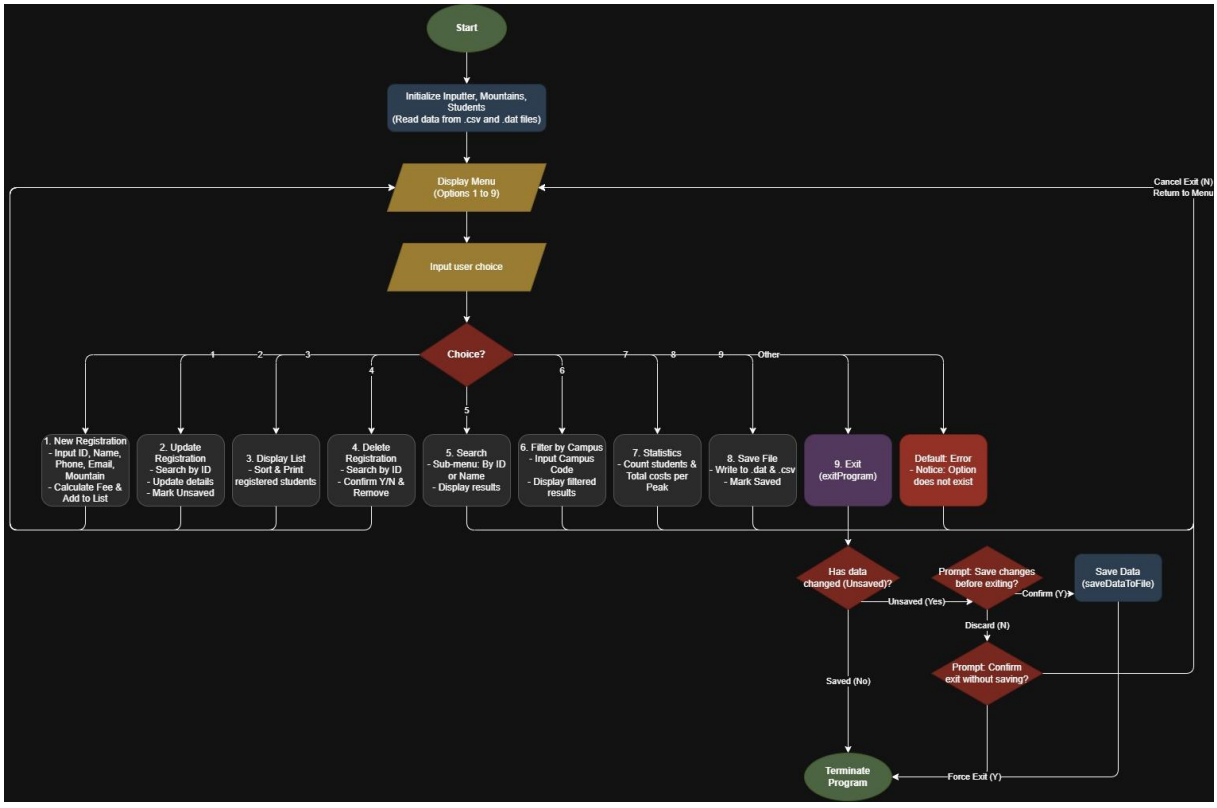
```
9. Exit
======================================================================
Your choice: 9
You have unsaved changes. Do you want to save before exiting? (Y/N): N
Are you sure you want to exit without saving? (Y/N): N

============== MOUNTAIN HIKING CHALLENGE REGISTRATION ==============
```

(Insert image: Exit screen - prompt to save when there is unsaved data)

5. Overall Workflow

5.1. Program Overall Flowchart

The flowchart below describes the overall workflow of the program from start to finish. The program operates in a menu-driven model (loop), meaning after completing a feature, the system returns to the main menu until the user selects Exit.



(Insert image: Overall Flowchart - Complete program workflow)

Workflow explanation:

Start -> Load MountainList.csv into Mountains -> Load registrations.dat into Students (if file exists) -> Display Menu -> Read user choice -> Execute feature -> Return to Menu (loop) -> When Exit is selected -> Check unsaved data -> Prompt to save -> End.

5.2. Sequence Diagram - New Registration

The Sequence Diagram describes the interaction sequence between objects when performing the New Registration feature. Participating objects: Operator, Main, Inputter, Mountains, Students, Student.

Sequence Diagram explanation:

57. 1. Operator selects New Registration from the menu.
58. 2. Main calls Inputter to input and validate ID, Name, Phone, Email.
59. 3. Main calls Students.searchById() to check for duplicate ID.
60. 4. Main calls Mountains.isValidMountainCode() to validate the mountain code.
61. 5. Main creates a new Student object and calls calculateFee() to compute the fee.
62. 6. Main calls Students.add() to add the student to the list and marks unsaved.
63. 7. Main displays a success message to the Operator.

6. Sample Data

6.1. MountainList.csv File

The MountainList.csv file contains a list of 13 mountain peaks, loaded when the program starts. Structure: Code, Mountain, Province, Description.

Code	Mountain	Province	Description
1	Ham Rong Mountain	Lao Cai	Near Sa Pa center, highest point 1850m
2	Doi Bo Mountain	Lao Cai	
3	Pha Luong Mountain	Son La	Nearly 2000m, the Roof of Moc Chau
4	Hon Vuon Mountain	Hue	
5	Da Do Mountain	Ninh Thuan	
6	Da Bia Mountain	Phu Yen	
7	Chu Hreng Mountain	Kon Tum	
8	Lang Biang Mountain	Lam Dong	
9	Ta Nang Mountain	Lam Dong	
10	Cam Mountain	An Giang	
11	Thi Vai Mountain	Vung Tau	
12	Dinh Mountain	Vung Tau	
13	Co Tien Mountain	Khanh Hoa	Has 3 peaks; conquer peak 1 only

6.2. registrations.csv File

The registrations.csv file is exported when the user selects Save Data. This is a text file that can be opened with Excel.

StudentID	Name	Phone	Email	MountainCode	TuitionFee
SE203056	Nguyen Phu Khuong	0363561629	khuong@gmail.com	1	3,900,000

7. Data Validation Rules

The system applies strict input data validation using Regular Expressions (Regex). The table below summarizes the validation rules for each data field.

Field	Rule	Valid Examples	Invalid Examples
Student ID	2-char campus code (SE/HE/DE/QE/CE) + 6 digits	SE203056 HE180001	AB123456 SE12345 SE1234567
Name	2-20 characters, letters and spaces only	Nguyen Van A Le Thi B	A 12345 Name longer than 20 chars abc
Phone	10 digits, starts with 0	0363561629 0912345678	123456789 0123 abcdefghij
Email	user@domain.ext (letters, digits, +, _, ., -)	abc@gmail.com test_1@fpt.edu.vn	@gmail.com abc@ abc.gmail.com
Mountain Code	Must exist in the MountainList.csv list (1-13)	1 5 13	0 14 abc
Campus Code	SE, HE, DE, QE, or CE (case-insensitive)	SE he DE	AB SF 123
Menu Choice	Integer from 1 to 9	1 5 9	0 10 abc
Confirm (Y/N)	Y or N (case-insensitive)	Y n	yes no 1

Registration fee calculation rules:

Carrier	Prefix Codes	Registration Fee
Viettel	032, 033, 034, 035, 036, 037, 038, 039, 096, 097, 098, 086	3,900,000 VND (35% discount)

VNPT	081, 082, 083, 084, 085, 088, 091, 094	3,900,000 VND (35% discount)
Other (Mobifone, ...)	Other prefix codes	6,000,000 VND (no discount)

8. Conclusion

8.1. Achievements

- Successfully completed all 9 features as required by the LAB211 assignment.
- Applied Object-Oriented Programming (OOP) with a clear structure of 9 classes/interfaces.
- Implemented strict input data validation using Regex; the program does not crash on invalid input.
- Automatic fee calculation based on phone carrier (Viettel/VNPT receive 35% discount).
- Reads mountain data from CSV file; saves registrations to binary file (.dat) and exports CSV.
- Smart exit handling: warns the user when there is unsaved data.
- Displays data in formatted tables with headers, dividers, and sorted by ID.
- Code exceeds 200 LOC with clear variable/method naming following conventions.

8.2. Applied Knowledge

- OOP: Class, Interface, Inheritance, Polymorphism, Encapsulation.
- Java Collections: ArrayList, HashMap, Collections.sort().
- Java I/O: BufferedReader, FileReader, ObjectInputStream/ObjectOutputStream, BufferedWriter.
- Serialization: Storing Java objects to binary file.
- Comparable: Sorting objects by custom criteria.
- Regex: Validating input data formats.
- Design Pattern: Menu-driven architecture, Separation of Concerns.

8.3. Limitations and Future Improvements

Limitations:

- No graphical user interface (GUI); the application is console-based only.
- No multi-language support (Vietnamese accented names may encounter encoding issues).
- No automated unit tests.

9. Feature Extension – Volunteer Management

9.1. Introduction to the New Feature

In this extended version, the Mountain Hiking Challenge Registration system is supplemented with the Volunteer Management module. The objectives of this extension are:

- Refactoring: Apply the Inheritance principle by creating an abstract class named Person as the common superclass for both Student and Volunteer, which helps to reuse source code and comply with OOP principles.
- Adding Volunteer module: Allows managing the list of volunteers supporting the mountain hiking trips, including CRUD operations and shift assignment functions (Assign to Shift).
- Extending Acceptable: Adding new regex patterns to validate the input data for Volunteer.
- Extending Main Menu: Add menu option 9 (Volunteer Management) with its own sub-menu.

New files added:

No.	File Name	Description
1	Person.java	Abstract class – common superclass
2	Skill.java	Enum – list of volunteer skills
3	Volunteer.java	Class – Volunteer entity
4	Volunteers.java	Class – manages the list of Volunteers

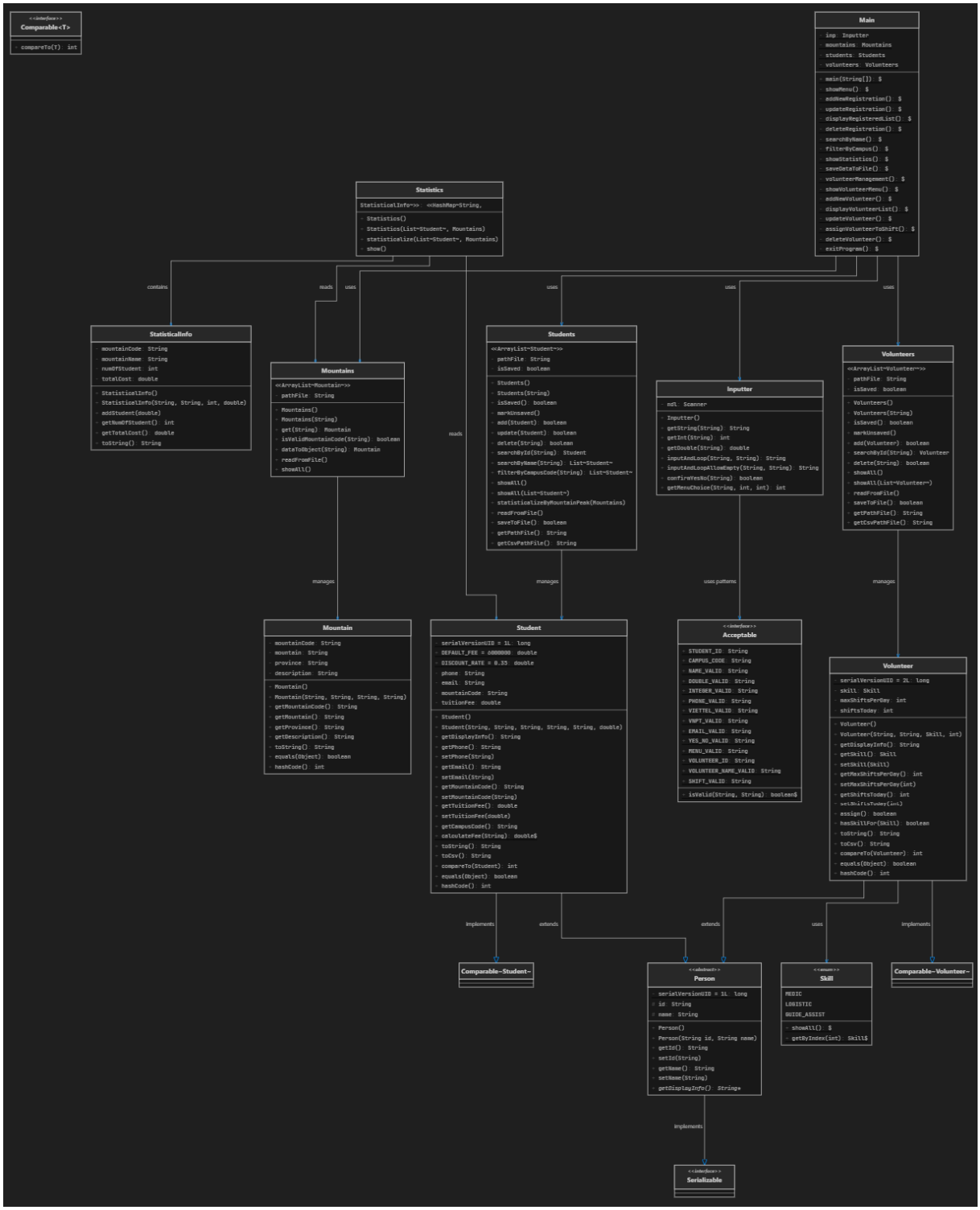
Modified files:

No.	File Name	Modifications
1	Student.java	Refactored: inherits from Person instead of declaring id and name directly
2	Acceptable.java	Added 3 new regex patterns for Volunteer validation
3	Main.java	Added Volunteer Management menu and its corresponding handling methods

9.2. Class Hierarchy Design (Inheritance – Abstract Class)

Prior to the extension, the Student class had the id and name attributes declared directly. When adding the Volunteer object (which also requires id and name), code duplication would be inevitable without refactoring.

Solution: Create an abstract class Person containing common attributes and methods, then have both Student and Volunteer inherit from Person.



(Please insert image: Overall System Class Diagram after refactoring – including all classes)

9.3. Description of New Classes

9.3.1. Abstract Class – Person

File: Person.java | Lines of code: 36 lines

Component	Description
abstract class Person	Abstract class, cannot be instantiated directly
implements Serializable	Allows serialization/deserialization of objects to save to the .dat file
protected String id, name	Common attributes, accessible from subclasses
abstract getDisplayInfo()	Abstract method – forces subclasses to override it

9.3.2. Enum – Skill

File: Skill.java | Lines of code: 20 lines

Value	Meaning
MEDIC	Volunteer with medical/first aid skills
LOGISTIC	Volunteer responsible for logistics and transportation
GUIDE_ASSIST	Volunteer assisting the tour guides

9.3.3. Class – Volunteer

File: Volunteer.java | Lines of code: 115 lines

Declared to inherit from Person (public class Volunteer extends Person implements Comparable<Volunteer>).

Attribute	Type	Description
skill	Skill	Specialized skill
maxShiftsPerDay	int	Maximum number of shifts per day (1–3)
shiftsToday	int	Number of shifts accepted for the current day
Method	**Description**	
assign()	Assign shift: increments shiftsToday by 1 if max is not reached. Returns true if successful.	
hasSkillFor(Skill)	Checks if the volunteer has the required skill. GENERAL slot accepts anyone.	
getDisplayInfo()	Overridden from Person – displays information in a formatted	

	table.
toCsv()	Converts the information to CSV format.

9.3.4. Class – Volunteers

File: Volunteers.java | Lines of code: 150 lines

Inherits from ArrayList<Volunteer> and manages the Volunteer list. Supports methods such as add, searchById, delete, showAll, readFromFile, saveToFile (saves to volunteers.dat and exports CSV).

9.4. Modifications to Existing Classes

9.4.1. Student (refactored)

Before (Old)	After (New)
implements Serializable, Comparable<Student>	extends Person implements Comparable<Student>
Declared private String id; private String name;	Inherits id and name from Person (protected)
Constructor: this.id = id; this.name = name;	Constructor: super(id, name);
No getDisplayInfo() method	Overrides getDisplayInfo() returning toString()

9.4.2. Acceptable (updated)

Pattern	Explanation
VOLUNTEER_ID	Starts with "VL" (case-insensitive) + 3 digits
VOLUNTEER_NAME_VALID	3–30 alphabetic characters (including Vietnamese) + spaces
SHIFT_VALID	Accepts values 1, 2, or 3 only

9.4.3. Main (updated)

Main Menu modifications: Expanded from 9 options to 10 options, adding "9. Volunteer Management".

```
============== MOUNTAIN HIKING CHALLENGE REGISTRATION ==============
1. New Registration
2. Update Registration Information
3. Display Registered List
4. Delete Registration Information
5. Search Participants by Name
6. Filter Data by Campus
7. Statistics of Registration Numbers by Location
8. Save Data to File
9. Volunteer Management
10. Exit
======================================================================
```

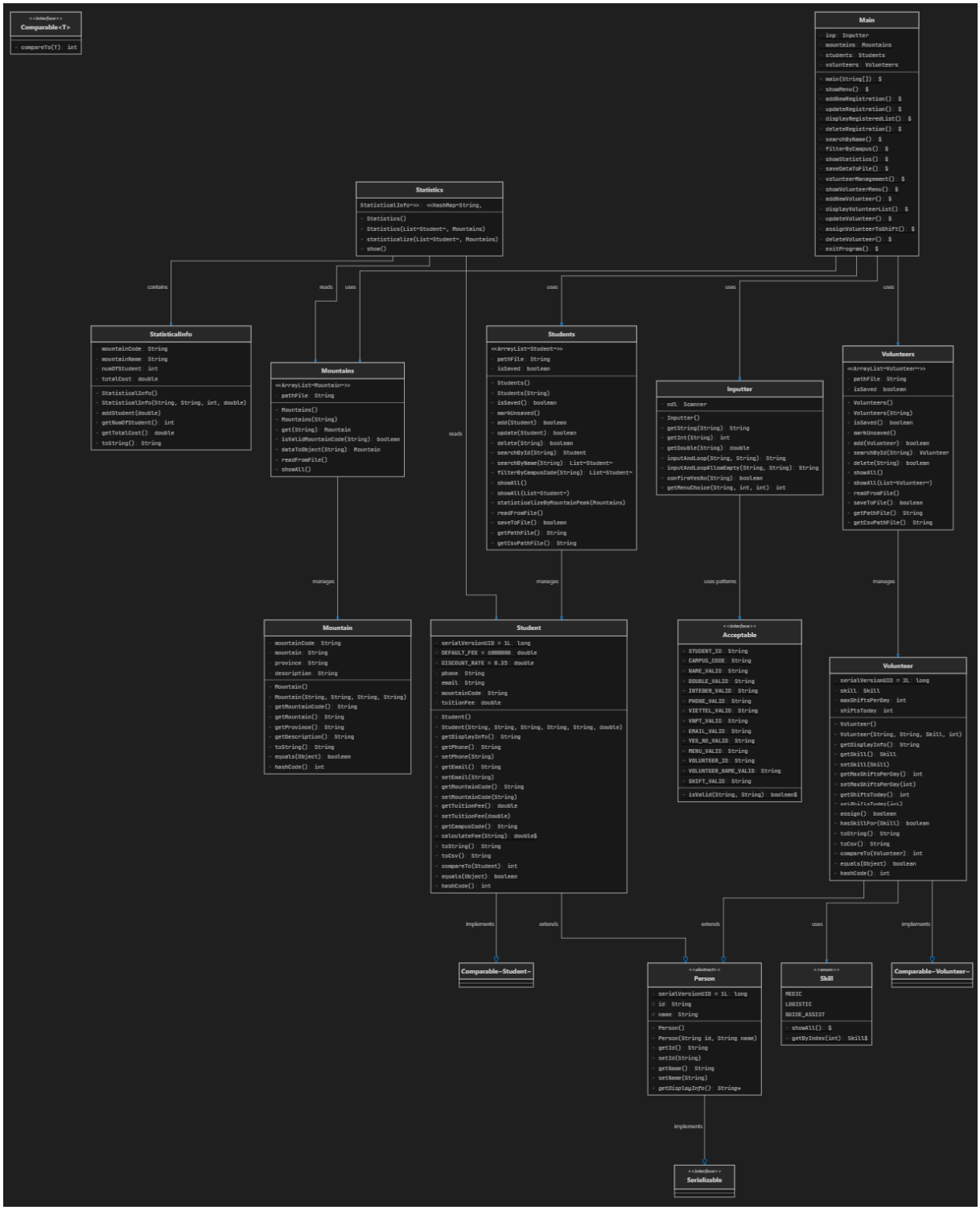
(Please insert image: Screenshot of Main Menu showing all 10 options)

Added Volunteer Management sub-menu with 6 features (Add, Display, Update, Assign, Delete, Back).

```
======================================================================
Your choice: 9

================ VOLUNTEER MANAGEMENT ================
1. Add New Volunteer
2. Display Volunteer List
3. Update Volunteer (Skill / Max Shifts)
4. Assign Volunteer to Shift
5. Delete Volunteer
6. Back to Main Menu
======================================================
Your choice:
```

(Please insert image: Screenshot of Volunteer Management sub-menu)

9.5. Updated Class Diagram



(Please insert image: Complete Class Diagram (PlantUML or Draw.io) – including all 13 classes/interfaces/enums)

9.6. Volunteer Management Features

9.6.1. Add New Volunteer

```
============================================================
Your choice: 1
---- Add New Volunteer ----
Volunteer ID [VL + 3 digits, e.g. VL001]: VL293
Name (3-30 chars): Phu Khuong
Select skill:
1. MEDIC
2. LOGISTIC
3. GUIDE_ASSIST
Skill (1-3): 3
Max shifts per day (1-3): 2
Volunteer added successfully.
VL293    | Phu Khuong                | GUIDE_ASSIST    | max=2 | today=0
```

(Please insert image: Screenshot of successfully adding a new Volunteer)

```
============================================================
Your choice: 1
---- Add New Volunteer ----
Volunteer ID [VL + 3 digits, e.g. VL001]: VL293
Volunteer ID already exists. Please try again.
Volunteer ID [VL + 3 digits, e.g. VL001]:
```

(Please insert image: Screenshot of duplicate ID entry – displaying error message)

9.6.2. Display Volunteer List

```
======================================================
Your choice: 2
---- Volunteer List ----
-------------------------------------------------------------------------------
ID       | Name                    | Skill          | Max   | Today
-------------------------------------------------------------------------------
VL293    | Phu Khuong              | GUIDE_ASSIST   | max=2 | today=0
VL299    | Tran Van Nam            | MEDIC          | max=1 | today=0
-------------------------------------------------------------------------------
```

(Please insert image: Screenshot of the full Volunteer list)

9.6.3. Update Volunteer

```
======================================================
Your choice: 3
---- Update Volunteer ----
Volunteer ID to update: VL299
Current information:
VL299    | Tran Van Nam              | MEDIC          | max=1 | today=0
Press Enter to keep the old value.
Current skill: MEDIC
Select new skill (Enter to keep):
1. MEDIC
2. LOGISTIC
3. GUIDE_ASSIST
Skill (1-3, Enter to keep): 3
New max shifts/day (current: 1, 1-3, Enter to keep): 3
Updated successfully.
VL299    | Tran Van Nam              | GUIDE_ASSIST   | max=3 | today=0
```

(Please insert image: Screenshot of Update Volunteer process – showing old and new information)

9.6.4. Assign Volunteer to Shift

```
==========================================================
Your choice: 4
---- Assign Volunteer to Shift ----
Volunteer ID: VL299
Volunteer info: VL299    | Tran Van Nam             | GUIDE_ASSIST   | max=3 | today=0
Slot type:
1. GENERAL (any skill)
2. MEDIC (requires MEDIC skill)
Slot type (1-2): 1
Assigned successfully! Shifts today: 1/3
```

(Please insert image: Screenshot of successful Assignment)

```
==========================================================
Your choice: 4
---- Assign Volunteer to Shift ----
Volunteer ID: VL293
Volunteer info: VL293    | Phu Khuong               | GUIDE_ASSIST   | max=2 | today=0
Slot type:
1. GENERAL (any skill)
2. MEDIC (requires MEDIC skill)
Slot type (1-2): 2
This volunteer does not have MEDIC skill. Cannot assign to MEDIC slot.
```

(Please insert image: Screenshot of insufficient skill for MEDIC slot error)

9.6.5. Delete Volunteer

```
Your choice: 5
---- Delete Volunteer ----
Volunteer ID to delete: VL299
Volunteer details:
VL299    | Tran Van Nam             | GUIDE_ASSIST   | max=3 | today=1
Are you sure you want to delete this volunteer? (Y/N): Y
Volunteer has been successfully deleted.
```

(Please insert image: Screenshot of deleting Volunteer – confirming with Y)

```
============================================================
Your choice: 5
---- Delete Volunteer ----
Volunteer ID to delete: VL293
Volunteer details:
VL293    | Phu Khuong                | GUIDE_ASSIST    | max=2 | today=0
Are you sure you want to delete this volunteer? (Y/N): N
Deletion cancelled.
```

(Please insert image: Screenshot of cancelling deletion – pressing N)

9.7. Volunteer Data Storage Integration

The saveDataToFile() method was updated to save both Student (registrations.dat) and Volunteer (volunteers.dat) data.

```
Your choice: 10
You have unsaved changes. Do you want to save before exiting? (Y/N): Y
---- Save Data to File ----
Registration data has been successfully saved to `registrations.dat`.
CSV report has been exported to `registrations.csv`.
Volunteer data has been successfully saved to `volunteers.dat`.
Volunteer CSV has been exported to `volunteers.csv`.
Goodbye!
```

(Please insert image: Screenshot of Save Data result – showing both Student and Volunteer saved successfully)

Exit Program: Checks both data sources. Warns if any data is unsaved.

```
====================================================================
Your choice: 10
You have unsaved changes. Do you want to save before exiting? (Y/N): N
Are you sure you want to exit without saving? (Y/N): Y
Goodbye!
```

(Please insert image: Screenshot of Exit prompt when there are unsaved changes)

9.8. Conclusion of the Extension

The Volunteer Management extension has successfully:

- Applied Inheritance: Created abstract class Person as the superclass.
- Applied Polymorphism: getDisplayInfo() is overridden in each subclass.
- Applied Encapsulation: Utilized Skill enum, and Volunteers class to manage the list.
- Integrated fully and operated smoothly with the existing Student system.

The total lines of code after the extension reached 1,518 lines with 13 class/enum/interface files.

--- End of Report ---